

RAPPORT D'ANALYSE

Projet audité : HeyDo

Critère 1 – Fonctionnalité et respect des User Stories (Note : 8/20)

Constat général : Un gros manque de méthodologie

Le premier problème qu'on a rencontré, c'est que le dépôt GitHub du projet ne fournit aucun document de User Stories, aucun cahier des charges et aucun modèle de base de données (MEA). C'est un vrai manque méthodologique pour une SAE. Comme on ne peut pas vérifier si les fonctionnalités codées correspondent à ce que le groupe s'était fixé, on a décidé de les évaluer par rapport au besoin global exprimé par notre client commun (M. Patrice Bringuier) lors de nos échanges en S3.

Une application locale, pas réseau

Le client demandait clairement un système capable de piloter à distance plusieurs lecteurs audio connectés sur un réseau. Alors que, ce qui a été livré est une simple démo qui ne tourne qu'en local sur un seul PC. Pire encore, l'ID du lecteur est fixé à "1" en dur dans le code. Si on branche un deuxième ordinateur sur le réseau et qu'on essaie de le connecter, c'est impossible car le système ne connaît que le lecteur numéro 1. Il n'y a donc aucune vraie communication réseau, ce qui casse l'un des points les plus importants du projet.

Des fonctionnalités clés totalement absentes :

→ **Transfert des fichiers** : Le cahier des charges exigeait que les fichiers MP3 soient transférés du serveur vers les lecteurs à travers le réseau. Ici, rien ne bouge, aucun fichier n'est transféré.

→ **Génération des M3U** : Le planning du service marketing devait générer un fichier playlist (M3U). Dans ce projet, les playlists restent bloquées dans la base de données et ne sont jamais exportées pour être lues par un vrai lecteur audio.

→ **Lecture audio simulée** : Quand on lance une musique, au lieu que le son sorte d'un haut-parleur physique, c'est juste un nouvel onglet Chrome qui s'ouvre sur le PC du développeur.

→ **Continuité sans internet** : Le client insistait pour que la musique continue de jouer grâce à des MP3 stockés localement en cas de coupure réseau. Ce mécanisme de sécurité n'a pas du tout été pensé.

→ **Gestion des pannes** : Le statut "KO" d'un lecteur s'affiche bien, mais aucune alerte automatique n'est envoyée. Il faut que l'administrateur ait les yeux fixés sur l'écran au bon moment pour s'en rendre compte.

Ce qui marche quand même : L'authentification avec les rôles (Admin, Marketing, Commercial) fonctionne bien.

Critère 2 – Qualité du Code (Note : 11/20)

Le code est généralement lisible dont il respecte la structure MVC, et le choix de technologies est logique et acceptable. Mais pour un projet de fin de semestre, on a quelques remarques :

→ **Un code "brouillon"** : Beaucoup de lignes de débogage et de commentaires inutiles ont été laissés dans les fichiers. Ça nuit à la lisibilité.

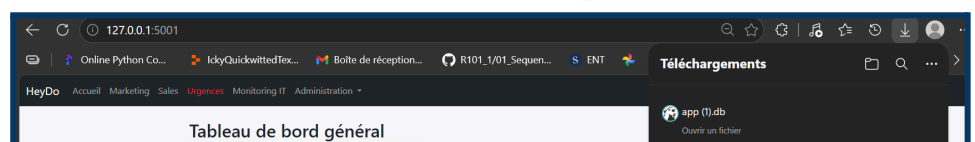
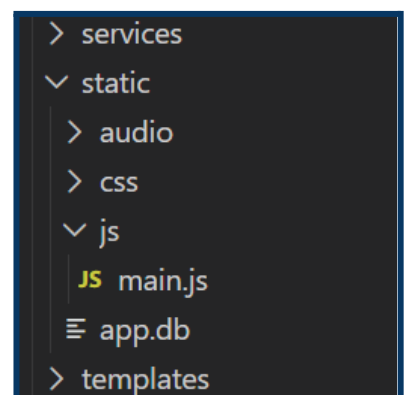
→ **Historique Git très faible** : Seulement 16 commits sur 3 mois de projet ! Git est censé raconter l'histoire du développement. Un historique aussi vide montre que l'outil n'a pas été utilisé régulièrement.

→ **Zéro commentaire technique** : Les fonctions et les classes ne sont pas du tout documentées. Pour comprendre ce que fait une fonction, on est obligé de lire tout son code.

Critère 3 – Gestion des Données (Note : 8/20)

La base de données existe et gère le minimum, mais elle souffre d'une faille de sécurité critique et de gros oublis :

→ **Une faille de sécurité énorme** : La base de données est stockée dans un dossier "static" public du serveur web. Concrètement, si quelqu'un tape l'adresse du serveur suivie de /static/app.db, il peut télécharger toute la base de données (avec les mots de passe, les utilisateurs, etc.). C'est une règle de base : une BDD ne doit jamais être accessible publiquement.



→ **Pas de logs** : Le client voulait garder une trace (un historique) de toutes les diffusions. Rien n'a été fait de ce côté-là.

→ **Paramétrage codé en dur** : Les plages horaires matin/après-midi sont stockées dans un simple fichier texte d'une seule ligne, qu'on ne peut pas modifier depuis l'interface web. Pour changer l'heure, l'administrateur doit aller fouiller dans les fichiers du serveur.

```
SAE301-main > {} scheduler_config.json > ...  
1  {"is_active": false, "matin": 1, "apres_midi": 1}
```

Critère 4 – Interface Utilisateur (Note : 11/20)

L'interface est visuellement correcte, mais pour une application qui est censée faire de la "supervision", il y a des problèmes importants liés au confort des utilisateurs. :

→ **Pas de temps réel** : Le tableau de bord ne se rafraîchit pas tout seul. Si un lecteur tombe en panne à 14h05, l'administrateur ne le verra que s'il recharge la page manuellement (ce qui peut arriver plusieurs heures plus tard). Et comme aucune alerte sonore ou visuelle ne s'affiche, c'est très peu pratique.

→ **Blocage sur les alertes urgentes** : Si le service commercial lance une alerte urgente par erreur, il n'y a aucun bouton pour l'annuler sur l'interface. Il faut obligatoirement appeler le développeur pour qu'il aille arrêter le processus directement dans les fichiers du serveur.

→ **Des menus déroulants vides** : On a remarqué des listes déroulantes (comme le choix de l'annonce commerciale à diffuser) qui ne contiennent aucune option. C'est donc impossible de lancer l'annonce (voir photo).



→ **Des boutons "fantômes"** : Plusieurs boutons sont cliquables mais ne déclenchent aucune action.

→ **Difficile à lancer** : Pour lancer l'application, il faut ouvrir 3 fenêtres de terminal en même temps.

Critère 5 – Installation & Déploiement (Note : 13/20)

C'est le point le plus acceptable et le moins mauvais du projet. D'abord le README est clair et les dépendances sont bien listées avec leurs versions. Ainsi la BDD se crée toute seule au lancement. Cependant un point mérite d'être soulevé :

→ **Utilisation d'un système windows au lieu de linux**: le CDC précise explicitement que le serveur doit tourner sous Linux. Or le projet fournit un script de démarrage au format .bat, qui ne fonctionne que sur Windows. C'est une contradiction directe avec les exigences techniques du cahier des charges.

CONCLUSION

Après avoir discuté de ces 5 critères avec tous les membres de notre groupe, le projet HeyDo obtient une note globale de 51/100.

Cette note reflète un écart beaucoup trop grand entre les attentes du cahier des charges et ce qui a réellement été codé. Le plus gros problème, c'est que la partie "Réseau" (qui est censée être le cœur absolu de cette SAE) est totalement absente. Reprendre ce projet pour l'améliorer nous obligerait à casser presque toute leur architecture pour reconstruire un vrai système client/serveur fonctionnel.

Décision finale : Notre groupe a donc pris la décision de ne pas reprendre le projet réalisé par l'équipe HeyDo.